CPE 486/586: Deep Learning for Engineering Applications

07 Variational AutoEncoder

Spring 2026

Rahul Bhadani

Outline

Motivation

Estimating Probability Distributions for Generative Processes

Given $\mathbf{x}$, we aim to estimate $P(\mathbf{x})$ to provide a complete description of the probabilistic model that generates the observed data.

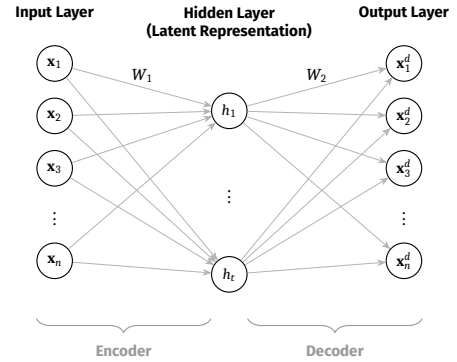
- **Data Synthesis:** We want to learn generative processes capable of producing new datasets.
- **Noise Reduction:** By discarding noise, we can more effectively learn underlying physical laws.
- **Causality:** Generative models provide a framework to express and understand causal relations.

> ***Example:*** *If we understand the generative process of wildfires, we can apply that knowledge across different regions, such as California or Australia.*

In this lecture, we model generative processes using Variational Autoencoders (VAEs).

AutoEncoder

What is AutoEncoder



Loss function is generally reconstruction loss:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n} ||\mathbf{x}_i - \mathbf{x}_i^d||_2^2$$

An **autoencoder** is a neural network, whose main job is to encode the input into a compressed representation, and then decode it back such that the reconstructed input is similar as possible to the original one.
It has three parts:

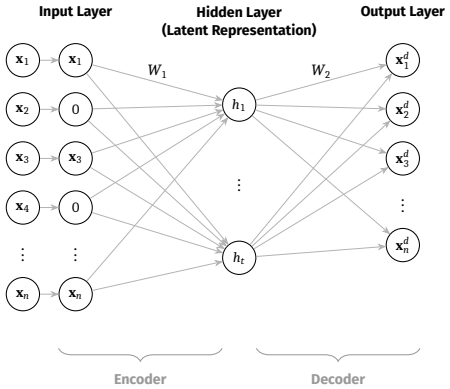
1. Encoder: Learns the most relevant features and transfers data into a latent space.
2. Latent representation: a middle layer that represents feature in some low dimensional space.
3. Decoder: it takes the reduced features from latent space and lift to the original dimension.

As it turns out, AutoEncoder is a neural network with a bottleneck.

Denoising Autoencoder

An AutoEncoder can end up learning an identity function (also called null function) where input=output.
To remedy, we add some noise to the input to corrupt it before feeding to AutoEncoder. Such an AE is called denoising AutoEncoder (DAE).
There are two ways to add noise to DAE:

1. Add Gaussian noise to the input.
2. Randomly set some input nodes to zero.
3. Masking out a certain percentage of the input's pixels in the case of image data.

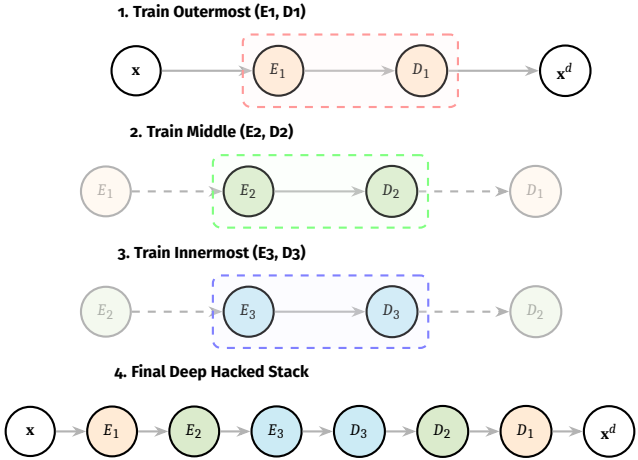


Stacked Autoencoders

The stacked autoencoder is a *hack* to obtain a deep model by training only shallow ones in a **layer-wise, greedy fashion**.

- **Phase 1:** Train the outermost encoder and decoder as a single shallow AE.
- **Phase 2:** Encode the dataset; use the result to train the next inner shallow AE.
- **Phase 3:** Repeat until reaching the innermost layers.
- **Result:** Stack the pre-trained layers into one deep architecture.

Visualizing the Greedy Training Process



Variational Inference

Motivation

We are interested in **posterior distribution calculation**.

Recall: Bayesian Framework

Given the parameter θ for a data distribution, we have prior as $p(\theta)$, a likelihood, if we write dataset as $\mathcal{D}$, then the posterior distribution can be calculated as:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

with $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta$ as the *marginal distribution*.

θ denotes latent parameters that we want to infer.

So, what's the problem?

Motivation

We are interested in **posterior distribution calculation**.

Recall: Bayesian Framework

Given the parameter θ for a data distribution, we have prior as $p(\theta)$, a likelihood, if we write dataset as $\mathcal{D}$, then the posterior distribution can be calculated as:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

with $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta$ as the *marginal distribution*.

θ denotes latent parameters that we want to infer.

So, what's the problem?

Computing $p(\mathcal{D})$ is **hard**, **intractable** due to the volume of data needed, and hence as a result, computing the posterior is hard too.

Variational Inference (VI) is one way to do so.

Variational Objective

We turn VI into an optimization problem: we choose a family of tractable distributions $\mathcal{Q}$ and find a member $q(\theta) \in \mathcal{Q}$ that is as close to $p(\theta|\mathcal{D})$ as possible.

What's the criteria for measuring the closeness between two distributions?

Variational Objective

We turn VI into an optimization problem: we choose a family of tractable distributions $\mathcal{Q}$ and find a member $q(\theta) \in \mathcal{Q}$ that is as close to $p(\theta|\mathcal{D})$ as possible.

What's the criteria for measuring the closeness between two distributions?

KL-Divergence, JS-divergence, etc.

Goal: Minimize KL divergence from $q(\theta)$ to $p(\theta|\mathcal{D})$

KL Divergence I

Definition

$$KL(q(\theta)||p(\theta|\mathcal{D})) = \int q(\theta) \log \frac{q(\theta)}{p(\theta|\mathcal{D})} d\theta$$

But KL divergence still depends on $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)\theta$.
To see, how, we can rewrite the KL integral using the definition of Expectation.

KL Divergence II

$$
\begin{aligned}
\mathrm{KL}(q(\theta)||p(\theta|\mathcal{D})) &= \mathbb{E}_q\left[\log \frac{q(\theta)}{p(\theta|\mathcal{D})}\right] \\
&= \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta|\mathcal{D}) \\
&= \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q\left[\log \frac{p(\theta,\mathcal{D})}{p(\mathcal{D})}\right] \\
&= \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta,\mathcal{D})] + \mathbb{E}_q[\log p(\mathcal{D})] \\
&= \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta,\mathcal{D})] + \int q(\theta)\log p(\mathcal{D})d\theta \\
&= \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta,\mathcal{D})] + \log p(\mathcal{D})
\end{aligned}
$$

The last step is because $\log p(\mathcal{D})$ is a constant with respect to $q(\theta)$.

KL Divergence III

We can ignore $\log p(\mathcal{D})$ in our optimization problem.

Minimizing a quantity means maximizing its negative, and so we maximize the following quantity:

$$\begin{aligned}
\text{ELBO}(q) &= -(\text{KL}(q(\theta)||p(\theta|\mathcal{D})) - \log p(\mathcal{D})) \\
&= -\Big(\mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta, \mathcal{D})] + \log p(\mathcal{D}) - \log p(\mathcal{D})\Big) \\
&= \mathbb{E}_q[\log p(\theta, \mathcal{D})] - \mathbb{E}_q[\log q(\theta)]
\end{aligned}$$

We could expand joint probability and rewrite

KL Divergence IV

$$
\begin{aligned}
\text{ELBO}(q) &= \mathbb{E}_q[\log p(\mathcal{D}|\theta)] + \mathbb{E}_q[\log p(\theta)] - \mathbb{E}_q[\log q(\theta)] \\
&= \mathbb{E}_q[\log p(\mathcal{D}|\theta)] + \mathbb{E}_q\left[\log \frac{p(\theta)}{q(\theta)}\right] \\
&= \mathbb{E}_q[\log p(\mathcal{D}|\theta)] - \mathbb{E}_q\left[\log \frac{q(\theta)}{p(\theta)}\right] \\
&= \mathbb{E}_q[\log p(\mathcal{D}|\theta)] - \text{KL}(q(\theta)||p(\theta))
\end{aligned}
$$

We could also note that

$p(\mathcal{D}, \theta) = p(\mathcal{D})p(\theta, \mathcal{D})$ from the Bayes' rule.

KL Divergence V

Hence,

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\theta|\mathcal{D}) + \log p(\mathcal{D})] - \mathbb{E}_q[\log q(\theta)]$$

Taking constant out of the expectation, and distributing the expectation

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\theta|\mathcal{D})] - \mathbb{E}_q[\log q(\theta)] + \log p(\mathcal{D})$$

Since $\mathcal{D}$ is fixed , $\log p(\mathcal{D})$ is a constant with respect to q

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\theta|\mathcal{D})] - \mathbb{E}_q[\log q(\theta)] + \textit{constant}$$

Variational Objective: The ELBO

The **Evidence Lower Bound (ELBO)** is defined as:

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\mathcal{D}|\theta)] - \text{KL}(q(\theta) \parallel p(\theta))$$

because the marginal is called *evidence* as well.
A $\log p(\mathcal{D})$ is constant wrt q, hence minimizing $KL(q(\theta)||p(\theta|\mathcal{D}))$ is equivalent to maximizing ELBO.
ELBO consists of two terms:

- **Expected log-likelihood:** Encourages q to place mass on parameters that explain the data well.
- **Negative KL to prior:** Encourages q to be close to the prior (regularizer).

VI maximizes a lower bound on the log marginal likelihood.

Question !

Show that $\log p(\mathcal{D}) \geq \text{ELBO}(q)$.

Question !

Show that $\log p(\mathcal{D}) \geq \text{ELBO}(q)$.

PROOF. —

$$\text{ELBO}(q) = -(\text{KL}(q(\theta)||p(\theta|\mathcal{D})) - \log p(\mathcal{D}))$$
$$\log p(\mathcal{D}) = \text{ELBO}(q) + (\text{KL}(q(\theta)||p(\theta|\mathcal{D}))$$
$$\log p(\mathcal{D}) \geq \text{ELBO}(q)$$

□

Maximizing ELBO finds approx distribution $p(\theta)$ for latent parameters θ that allows data to be predicted really well, but penalty occurs when $q(\theta)$ is far away from the prior $p(\theta)$.

What is Variational in VI?

We are not looking to find a single best value that maximizes the log likelihood, but rather a single best function.

Moving Beyond MLE

- We are **not** looking to find a single best value that maximizes the log-likelihood.
- Instead, we seek to identify the single **best function** (distribution).

The Methodology

- To achieve this, we employ **variational calculus**.
- The term "Variational" is derived directly from this mathematical framework.

Goal: Approximate a complex posterior by finding the closest member of a simpler family of distributions.

Variational Calculus, Briefly

Variational calculus (Calc is concerned with finding the functions that minimize or maximize quantities expressed as integrals. **[Pattern Recognition and Machine Learning (Christopher M. Bishop), Section 10.1]**

Unlike basic calculus, which focuses on derivatives of functions, variational calculus focuses on finding the function itself that produces the optimal outcome.

In Variational Calculus,

Find a function $y(x)$ that minimizes the functional $J[y] = \int F(x, y, y')dx$.
Here, $J[y]$ is not just a number but a **functional** — a mapping from functions to numbers.

The integral might represent things like total energy, time, or distance and our goal is to minimize it.

Some examples where Variational Calculus are useful

1. What is the shortest or smoothest path to the goal in case of autonomous navigation?
2. How do navigate while consuming the least energy?
3. How to avoid collision while being efficient?

The Euler-Lagrange Equation

The main tool in variational calculus is the Euler-Lagrange equation

$$\frac{d}{dx}\frac{\partial F}{\partial y'} - \frac{\partial F}{\partial y} = 0$$

which tells us the condition a function $y(x)$ mut satisfy to minimize/maximize a given functional. In robotics, $y(x)$ could be robot's trajectory. F could be energy, cost, control effort, distance, etc.

Mean-field Variational Family

The next question, how do we choose Q. A common choice for the variational family Q is the **mean-field family**. Other options are *full-covariance Gaussians*, and *Normalizing Flows*.

The mean-field variational family assumes that the latent variables are independent of each other in the approximate posterior. Mathematically, if we have parameters $\theta = (\theta_1, \theta_2, \ldots, \theta_M)$, the mean-field approximation factorizes as:

$$q(\theta) = \prod_{j=1}^{M} q_j(\theta_j)$$

Each q_j is a marginal variational distribution for the j-th parameter or group of parameters. The key point is that there are no cross-terms or dependencies between different θ_j's in the approximation.

The term "mean-field" comes from physics, specifically from statistical mechanics. In physics, mean-field theory approximates a many-body problem by replacing interactions with an average or "mean" field. Similarly, in variational inference, we're approximating a complex posterior where variables are dependent by a simpler distribution where they're independent.

Mean-field Variational Inference with ELBO I

Recall: The ELBO Definition

$$\begin{aligned}\text{ELBO}(q) &= \mathbb{E}_q[\log p(\theta, \mathcal{D})] - \mathbb{E}_q[\log q(\theta)] \\ &= \int \prod_{i=j}^{M} q_j(\theta_j) \log p(\theta, \mathcal{D}) d\theta - \int \prod_{i=j}^{M} q_j(\theta_j)\end{aligned}$$

We optimize ELBO with respect to a single variational density $q_j(\theta_i)$ and keep others fixed.

Mean-field Variational Inference with ELBO II

Derivation for q_j

$$
\begin{aligned}
\text{ELBO}(q_j) &= \int \prod_{i=j}^{M} q_j(\theta_j) \log p(\theta, \mathcal{D}) d\theta - \int \prod_{i=j}^{M} q_j(\theta_j) \log \prod_{i=j}^{M} q_j(\theta_j) d\theta \\
&= \int \prod_{i=j}^{M} q_j(\theta_j) \log p(\theta, \mathcal{D}) d\theta - \int q_j(\theta_j) \log q_j(\theta_j) d\theta_j - \underbrace{\int \prod_{i \neq j}^{M} q_j(\theta_j) \log \prod_{i \neq j}^{M} q_j(\theta_j) d\theta_{-j}}_{\text{Fixed with respect to } q_j \theta_j} \\
&\propto \int \prod_{i=j}^{M} q_j(\theta_j) \log p(\theta, \mathcal{D}) d\theta - \int q_j(\theta_j) \log q_j(\theta_j) d\theta_j \\
&= \int q_j(\theta_j) \Big(\int \prod_{i \neq j}^{M} q_j(\theta_j) \log p(\theta, \mathcal{D}) d\theta_{-j} \Big) d\theta_j - \int q_j(\theta_j) \log q_j(\theta_j) d\theta_j \\
&= q_j(\theta_j) \mathbb{E}_{q(\theta_{-j})} [\log p(\theta, \mathcal{D})] d\theta_j - \int q_j(\theta_j) \log q_j(\theta_j) d\theta_j
\end{aligned}
$$

Mean-field Variational Inference with ELBO III

We could use variational calculus to find the optimal variational density $q_j^*(\theta_j)$.
We can further define $\log \tilde{p}(\mathcal{D}, \theta_j) = \mathbb{E}_{q(\theta_{-j})}[\log p(\theta, \mathcal{D})] d\theta_j + \textit{Constant}$.
The constant is from the previous step just before the proportional.
Hence,

Relation to KL Divergence

$$\text{ELBO}(q_j) \propto \int q_j(\theta_j) \log \tilde{p}(\mathcal{D}, \theta_j) d\theta_j - \int q_j(\theta_j) \log q_j(\theta_j) d\theta_j$$

Merge the integral due to common integraion variable

$$= \int q_j(\theta_j) \log \frac{\tilde{p}(\mathcal{D}, \theta_j)}{q_j(\theta_j)} d\theta_j$$

$$= -\int q_j(\theta_j) \log \frac{q_j(\theta_j)}{\tilde{p}(\mathcal{D}, \theta_j)} d\theta_j$$

$$= -KL(q_j(\theta_j) || \tilde{p}(\mathcal{D}, \theta_j))$$ (Using the definition of KL Divergence)

The ELBO is equal to the negative KL divergence up to an additive constant.

Hence, maximizing the ELBO is equivalent to minimizing the KL Divergence between $q_j(\theta_j)$ and $\tilde{p}(\mathcal{D}, \theta_j)$.

Mean-field Variational Inference with ELBO IV

Hence, under the mean-field assumption, the optimal variational density is given by

The Optimal Mean-Field Update

$$q_j^* = exp\left(\mathbb{E}_{q(\theta_{-j})}[\log p(\theta, \mathcal{D})]d\theta_j + Constant\right)$$
$$= \frac{\mathbb{E}_{q(\theta_{-j})}[\log p(\theta, \mathcal{D})]d\theta_j)}{\int \mathbb{E}_{q(\theta_{-j})}[\log p(\theta, \mathcal{D})]d\theta_j}$$

Note that this is not an explicit solution because each optimal variational density depends on all others. Hence, the true solution will be iterative ones.
This process is called **Coordinated Ascent Variational Inference**.

Stochastic Gradients for Variational Inference

We can use gradient descent based methods for optimizing ELBO.

We need to compute gradients of the ELBO wrt to the parameters of q.

$$\mathcal{L}(\lambda) = \text{ELBO}(q_\lambda) = \mathbb{E}_{q_\lambda(\theta)}[\log p(\mathcal{D}|\theta)] - \text{KL}(q_\lambda(\theta) \parallel p(\theta))$$

where λ is the variational parameters such as mean and log-std of a Gaussian.

The KL term can often be computed analytically (e.g., when both q and p are Gaussians). The expected log-likelihood term, however, usually requires approximation.

In PyTorch, we can use reparameterization trick to estimate the gradients of the expectation.

Example: KL Divergence between Two Gaussians I

Derive the closed-form $KL(q\|p)$ where:

$$q(z) = \mathcal{N}(z;\, m,\, s^2) = \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{(z-m)^2}{2s^2}\right)$$

$$p(z) = \mathcal{N}(z;\, \mu_0,\, \sigma_0^2) = \frac{1}{\sqrt{2\pi\sigma_0^2}} \exp\left(-\frac{(z-\mu_0)^2}{2\sigma_0^2}\right)$$

Solution

$$\begin{aligned} KL(q\|p) &= \int_{-\infty}^{\infty} q(z)\, \log \frac{q(z)}{p(z)}\, dz \\ &= \int_{-\infty}^{\infty} q(z)\, \log q(z)\, dz - \int_{-\infty}^{\infty} q(z)\, \log p(z)\, dz \end{aligned}$$

Example: KL Divergence between Two Gaussians II

Taking the log of $q(z)$:

$$\begin{aligned}\log q(z) &= \log\left[\frac{1}{\sqrt{2\pi s^2}}\exp\left(-\frac{(z-m)^2}{2s^2}\right)\right] \\ &= -\frac{1}{2}\log(2\pi) - \frac{1}{2}\log\left(s^2\right) - \frac{(z-m)^2}{2s^2}\end{aligned}$$

Taking the log of $p(z)$:

$$\begin{aligned}\log p(z) &= \log\left[\frac{1}{\sqrt{2\pi\sigma_0^2}}\exp\left(-\frac{(z-\mu_0)^2}{2\sigma_0^2}\right)\right] \\ &= -\frac{1}{2}\log(2\pi) - \frac{1}{2}\log\left(\sigma_0^2\right) - \frac{(z-\mu_0)^2}{2\sigma_0^2}\end{aligned}$$

Example: KL Divergence between Two Gaussians III

Taking the difference:

$$\begin{aligned}\log q(z) - \log p(z) &= \left[-\frac{1}{2}\log(2\pi) - \frac{1}{2}\log\left(s^2\right) - \frac{(z-m)^2}{2s^2}\right] \\ &\quad - \left[-\frac{1}{2}\log(2\pi) - \frac{1}{2}\log\left(\sigma_0^2\right) - \frac{(z-\mu_0)^2}{2\sigma_0^2}\right] \\ &= \frac{1}{2}\log\left(\sigma_0^2\right) - \frac{1}{2}\log\left(s^2\right) - \frac{(z-m)^2}{2s^2} + \frac{(z-\mu_0)^2}{2\sigma_0^2} \\ &= \log\frac{\sigma_0}{s} - \frac{(z-m)^2}{2s^2} + \frac{(z-\mu_0)^2}{2\sigma_0^2}\end{aligned}$$

Example: KL Divergence between Two Gaussians IV

$$
\begin{aligned}
KL &= \mathbb{E}_q[\log q(z) - \log p(z)] \\
&= \mathbb{E}_q\left[\log \frac{\sigma_0}{s} - \frac{(z-m)^2}{2s^2} + \frac{(z-\mu_0)^2}{2\sigma_0^2}\right] \\
&= \log \frac{\sigma_0}{s} - \frac{1}{2s^2}\,\mathbb{E}_q\left[(z-m)^2\right] + \frac{1}{2\sigma_0^2}\,\mathbb{E}_q\left[(z-\mu_0)^2\right]
\end{aligned}
$$

Example: KL Divergence between Two Gaussians V

This is the variance of q:

$$\mathbb{E}_q\left[(z-m)^2\right] = s^2$$

Add and subtract m inside the square:

$$\begin{aligned}
\mathbb{E}_q\left[(z-\mu_0)^2\right] &= \mathbb{E}_q\left[(z-m+m-\mu_0)^2\right] \\
&= \mathbb{E}_q\left[(z-m)^2\right] + 2(m-\mu_0)\,\mathbb{E}_q[z-m] + (m-\mu_0)^2 \\
&= s^2 + 2(m-\mu_0)\cdot 0 + (m-\mu_0)^2 \\
&= s^2 + (m-\mu_0)^2
\end{aligned}$$

where $\mathbb{E}_q[z-m] = 0$ since m is the mean of q.

Example: KL Divergence between Two Gaussians VI

$$
\begin{aligned}
KL &= \log \frac{\sigma_0}{s} - \frac{1}{2s^2} \cdot s^2 + \frac{1}{2\sigma_0^2}\left[s^2 + (m-\mu_0)^2\right] \\
&= \log \frac{\sigma_0}{s} - \frac{1}{2} + \frac{s^2 + (m-\mu_0)^2}{2\sigma_0^2} \\
&= \log \frac{\sigma_0}{s} + \frac{s^2 + (m-\mu_0)^2}{2\sigma_0^2} - \frac{1}{2}
\end{aligned}
$$

Example: KL Divergence between Two Gaussians VII

Substituting $\mu_0 = 0$ and $\sigma_0 = 1$:

$$KL = \log \frac{1}{s} + \frac{s^2 + m^2}{2 \cdot 1} - \frac{1}{2}$$

$$= -\log s + \frac{s^2 + m^2}{2} - \frac{1}{2}$$

Variational AutoEncoder

Data Generating Process and a Neural Network

Two primary ways to generate:

- Probability Distribution Functions
- (Ordinary/Partial) Differential Equations

Variational Inference provides an elegant framework for modeling a generative process. But we always talked about a fixed form of PDF for data models (e.g. mean-field families).

However, for a very high-dimensional data, just thinking about a PDF of that scale is tiring.
Thus comes a Neural Network into the picture!

Why AutoEncoder is Insufficient?

A regular AutoEncoder maps $\mathbf{x} \in \mathcal{D} \mapsto \mathbf{z} \in \mathcal{Z}$, the latent space deterministically, which is not sufficient to model a generative process to get more synthentic dataset.

$\longmapsto$

What is Variational AutoEncoder?

Variational AutoEncoder (VAE) is a probabilistic framework that combines deep learning with Variational Inference.

Let's consider latent vector as $\mathbf{z}$ which provides the latent representation as seen earlier in AutoEncoder scheme.

The goal in Bayesian framework is to learn posterior $p(\mathbf{z}|\mathbf{x})$.

Steps in VAE

1. Use a neural network $f_{\mathbf{w}}$ parametrized by $\mathbf{w}$ to compute the posterior

$$q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})$$

2. Draw L random $\mathbf{z}$ vectors from the distribution.
3. Using another neural network $g_{\mathbf{v}}$ parameterized by $\mathbf{v}$ the Estimate $p_{\mathbf{v}}(\mathbf{x}|\mathbf{z})$.
4. Update $\mathbf{w}$ and $\mathbf{v}$ using gradient ascent/descent.

◇
Encoder

Maps data $\mathbf{x}$ to a distribution $q_{\mathbf{w}}(\mathbf{z}|x)$ over the latent space

⊙
Latent Space

A structured, continuous space where each point $\mathbf{z}$ represents a data concept

◇
Decoder

Maps a latent code $\mathbf{z}$ back to data space via $p_{\mathbf{v}}(\mathbf{x}|\mathbf{z})$

The presence of an encoder and decoder is what makes it an AutoEncoder.
Encoder is also called inference model. Decoder is called generative model.

Objective in VAE

We want to maximize $\log p_{\mathbf{v}}(\mathbf{x})$, the log-likelihood of our data. But as this is intractable and difficult, we optimize a lower bound instead through ELBO.

$$ELBO(\mathbf{w}, \mathbf{v}, \mathbf{x}) = \mathbb{E}_{q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})}[\log p_{\mathbf{v}}(\mathbf{x}|\mathbf{z})] - KL(q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))$$

which needs to be maxmized.

The Reparameterization Trick in VAE

To train VAE, we need gradients to flow back through the sampling operation. However, the sampling operation $z \sim q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})$ is not differentiable, as the randomness blocks the gradient. The flow of the information is disconnected due to randomness.

The Reparameterization Trick in VAE

To train VAE, we need gradients to flow back through the sampling operation. However, the sampling operation $z \sim q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})$ is not differentiable, as the randomness blocks the gradient. The flow of the information is disconnected due to randomness.

Hence, we have a reparameterization trick

Reparameterize the sample: separate the random part from the learned parameters:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

The Reparameterization Trick in VAE

To train VAE, we need gradients to flow back through the sampling operation. However, the sampling operation $z \sim q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})$ is not differentiable, as the randomness blocks the gradient. The flow of the information is disconnected due to randomness.

Hence, we have a reparameterization trick

Reparameterize the sample: separate the random part from the learned parameters:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Now, ε is a fixed random variable drawn from a standard normal distribution and the network parameters μ and σ are deterministic functions of $\mathbf{x}$, so gradients can flow through them, thus,

$$q_{\mathbf{w}}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), diag(\boldsymbol{\sigma}^2(\mathbf{x})))$$

The Reparameterization Trick in VAE

To train VAE, we need gradients to flow back through the sampling operation. However, the sampling operation $z \sim q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})$ is not differentiable, as the randomness blocks the gradient. The flow of the information is disconnected due to randomness.

Hence, we have a reparameterization trick

Reparameterize the sample: separate the random part from the learned parameters:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

Now, ε is a fixed random variable drawn from a standard normal distribution and the network parameters μ and σ are deterministic functions of $\mathbf{x}$, so gradients can flow through them, thus,

$$q_{\mathbf{w}}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), diag(\boldsymbol{\sigma}^2(\mathbf{x})))$$

For numerical stability, we output $\log \boldsymbol{\sigma}^2$ which can be any real number, $\boldsymbol{\sigma} > 0$ would require constraint.

The Reparameterization Trick in VAE

To train VAE, we need gradients to flow back through the sampling operation. However, the sampling operation $z \sim q_{\mathbf{w}}(\mathbf{z}|\mathbf{x})$ is not differentiable, as the randomness blocks the gradient. The flow of the information is disconnected due to randomness.

Hence, we have a reparameterization trick

Reparameterize the sample: separate the random part from the learned parameters:

$$\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I})$$

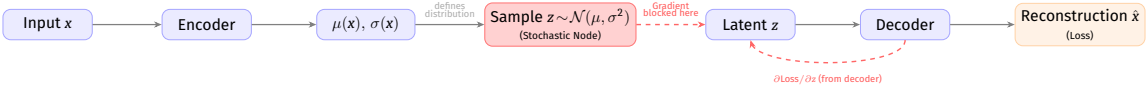
Now, ε is a fixed random variable drawn from a standard normal distribution and the network parameters μ and σ are deterministic functions of $\mathbf{x}$, so gradients can flow through them, thus,

$$q_{\mathbf{w}}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}(\mathbf{x}), diag(\boldsymbol{\sigma}^2(\mathbf{x})))$$

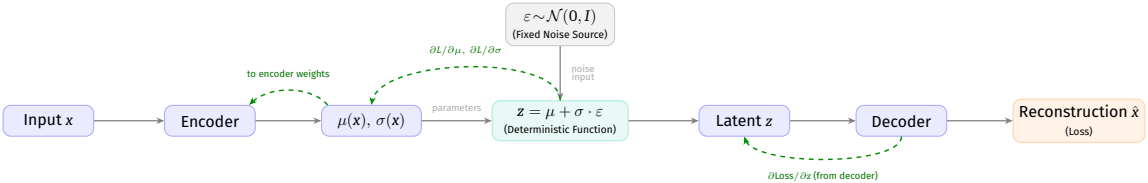
For numerical stability, we output $\log \boldsymbol{\sigma}^2$ which can be any real number, $\boldsymbol{\sigma} > 0$ would require constraint.

For this to work, each dimension of $\mathbf{z}$ is assumed **independent** making the covariance matrix diagonal. The encoder learns the mean and standard deviation (uncertainty) of each latent dimension.

The reparameterization Trick in VAE



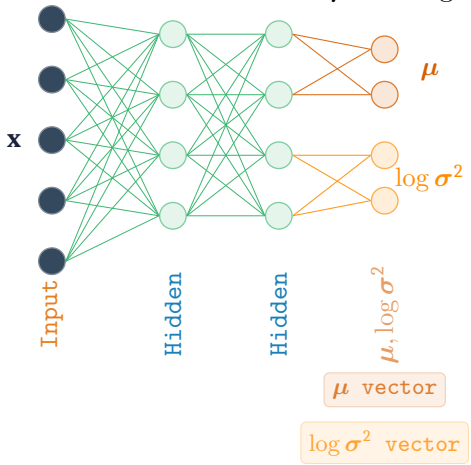
Before Reparameterization



After Reparameterization

Encoder in VAE

Thus, the encoder in VAE predict two set of values: μ and $\log \sigma^2$.



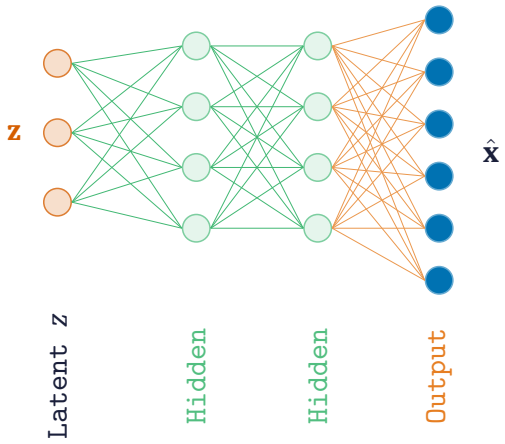
Decoder in VAE

Decoder models $p_{\mathbf{v}}(\mathbf{x}|\mathbf{z})$. Think that the Neural Network to model $p_{\mathbf{v}}$ is $g_{\mathbf{v}}$ that takes a latent code $\mathbf{z}$ and produces either reconstrucuted $\hat{\mathbf{x}}$ in deterministic fashion or a parameters of a distribution over $\mathbf{x}$ probabilistically.

For continuous data, we could model

$$p_{\mathbf{v}}(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}; g_{\mathbf{v}}(\mathbf{z}), \sigma_{\mathbf{x}}^2 \mathbf{I}$$

We can place the prior as $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$, that encourages the encoder to keep the latent representation centered and structured, enabling random generation: sample $\mathbf{z} \sim p(\mathbf{z})$ and decode.



Reconstruction Loss in VAE Objective

In ELBO, we could rewrite reconstruction loss

$$\mathbb{E}[\log p_{\mathbf{v}}(\mathbf{x}|\mathbf{z})] \approx -\frac{1}{2}||\mathbf{x} - \hat{\mathbf{x}}||^2$$

And interestingly, KL Divergence for two Gaussians have a closed form solution:

$$KL = -\frac{1}{2}\sum_{j=1}^{d}(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2)$$

Latent Space in VAE

On a trained VAE, after the convergence, the latent space has a desirable properties:

- Continuity, with which nearby points in latent space decode to similar outputs;
- Completeness, where any sampled $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ decodes to some plausible data thereby giving you a realistic looking synthetic data;
- Interpolation, where interpolating between two codes is semantically smooth;

A well trained VAE organizes its latent space in such a way that similar looking concepts cluster together.

A Note on Isotropic Gaussian in VAE

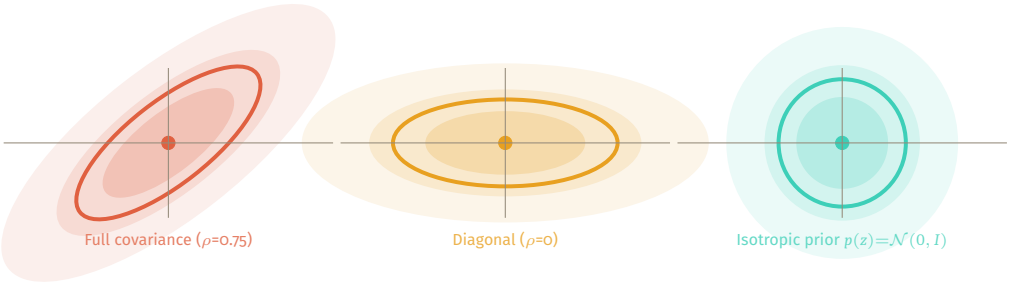
A Gaussain $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ is isotropic when $\Sigma = \sigma^2\mathbf{I}$, its probability contours are perfect hyperspheres, not eillipsoid.

Prior uses $p(\mathbf{z}) = \mathcal{N}(0, \mathbf{I})$ which is full isotropic.

Posterior is modeled using $q_{\mathbf{w}}(\mathbf{z}|\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}, diag(\boldsymbol{\sigma}^2))$ which is a diagonal Gaussian.

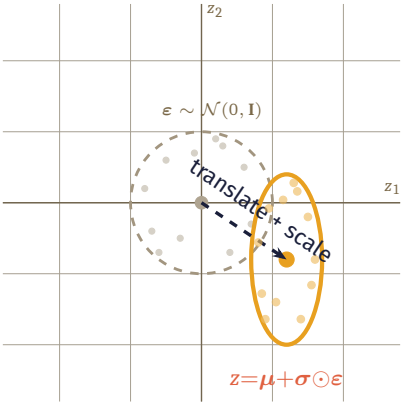
- The diagonal covariance means each dimension is independent, so the joint KL factors into a sum of per-dimension KLs. Each term has an exact formula, as seen earlier.
- With Isotropic Gaussain, drawing random $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ is just d independent standard normals.

Three Gaussians



Left: full covariance (tilted ellipse, correlated dims). Centre: diagonal (axis-aligned ellipse). Right: isotropic (circle — VAE prior).

Reparameterization



$\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ *(circle).* $\mathbf{z} = \mu + \sigma \odot \varepsilon$ *(axis-aligned ellipse). Dashed arrow: translate + scale, all differentiable.*

Reading List

1. Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P. A. (2008, July). Extracting and composing robust features with denoising autoencoders. In Proceedings of the 25th international conference on Machine learning (pp. 1096-1103). https://dl.acm.org/doi/pdf/10.1145/1390156.1390294
2. Alain, Guillaume, and Yoshua Bengio. "What regularized auto-encoders learn from the data-generating distribution." The Journal of Machine Learning Research 15, no. 1 (2014): 3563-3593. https://www.jmlr.org/papers/volume15/alain14a/alain14a.pdf
3. Vincent, Pascal, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion." Journal of machine learning research 11, no. 12 (2010).
4. Zhai, J., Zhang, S., Chen, J. and He, Q., 2018, October. Autoencoder and its various variants. In 2018 IEEE international conference on systems, man, and cybernetics (SMC) (pp. 415-419). IEEE.
5. Li, Pengzhi, Yan Pei, and Jianqiang Li. "A comprehensive survey on design and application of autoencoder in deep learning." Applied Soft Computing 138 (2023): 110176.
6. Michelucci, Umberto. "An introduction to autoencoders." arXiv preprint arXiv:2201.03898 (2022).
7. Bank, Dor, Noam Koenigstein, and Raja Giryes. "Autoencoders." Machine learning for data science handbook: data mining and knowledge discovery handbook (2023): 353-374.

Reading List

- Section 10.1 of Pattern Recognition and Machine Learning (Christopher M. Bishop 2006)
- Stochastic Variational Inference https://arxiv.org/pdf/1206.7051
-

Code

https://github.com/rahulbhadani/CPE487587_SP26/blob/master/Code/Chapter_07_Variational_Inference.ipynb

The End